

InventiX

Backend Integration Plan

How we connect the app to the real backend, one feed at a time, and how we check that each part works.

Project	InventiX — inventory app for Sri Lankan grocery shops
Team	Team 4
Git branch	<code>feature/backend-integration</code>
Written	11 September 2026
Target finish	14 September 2026

How to read this document. Section 1 explains where we are now. Section 2 lists the jobs to finish **before** we start. Sections 3 to 6 are the plan itself. Section 7 is the list of problems we already found. Sections 8 to 10 help when something breaks. Words that may be new are explained in the glossary at the end.

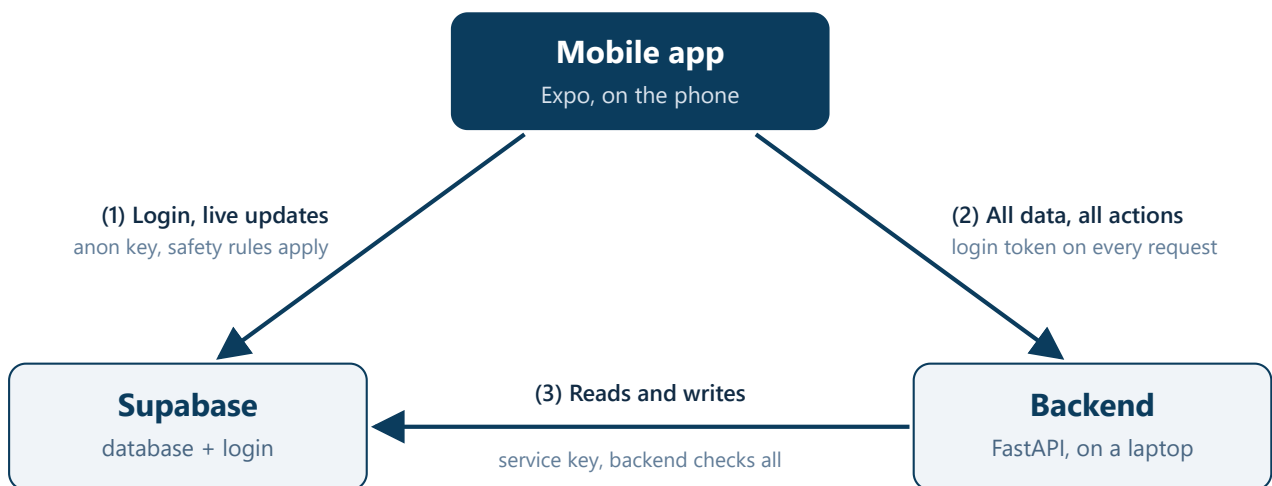
1. Where we are now

InventiX has three parts. All three are **already built**. What is left is to make them work together and check the results.

Part	Status	What it means
Database (Supabase)	DONE	All 18 tables, the security rules, 36 catalog products and the demo shop and supplier are in Supabase. One small step is still missing: switching on live updates (see section 2).
Backend (FastAPI, Python)	BUILT	All 43 endpoints exist and all 119 automatic tests pass. The login check was fixed on 11 September so it accepts our Supabase project's tokens.
Frontend (Expo app)	BUILT	Every screen exists. Until now it showed fake "fixture" data stored inside the app.
Settings (<code>.env</code> files)	DONE	Both <code>.env</code> files hold the right Supabase address and keys. The app knows where the backend is (<code>EXPO_PUBLIC_API_URL</code>).
Login	WORKS	Both demo accounts can sign in. The backend now accepts their tokens (to be confirmed in Phase 0, step 7).

How the three parts talk to each other

The app has **two** connections, not one. This is on purpose.



- **Connection 1 — App to Supabase.** Only for signing in, and for "live updates" (a screen refreshes by itself when data changes). This already works.
- **Connection 2 — App to Backend.** Every list, number and button action. The app sends the login token with each request, so the backend knows who is asking. **This is what we are integrating.**
- **Connection 3 — Backend to Supabase.** The backend reads and writes the real data. Already working.

One login, one token, two uses. When you sign in, Supabase gives the app a token. The app shows that same token to the backend on every request. The backend checks it, finds the user's role (shop or supplier), and only then answers. Login itself never needs the backend.

What "integration" means for us

The switch is already on. Because `EXPO_PUBLIC_API_URL` is set, the app stopped using its fake data and now asks the backend for everything. We do **not** need to delete any fixture code, because the app no longer runs it.

So integration means: **open each screen, check that the real data looks right and every button works, and fix anything that does not.** We do this one feed at a time so problems are easy to find.

2. Before we start — Phase 0 checklist

Finish these eight jobs first. Items 1 to 7 take about 30 minutes in total. Step 7 is the "go / no-go" test: if it passes, integration can begin.

#	Job	Why, and how
1	Commit the work done so far MUST	The login fix (<code>security.py</code>), its tests, the updated <code>frontend/.env.example</code> and the demo email change are not saved in git yet. Commit them so every feed starts from a clean, known point. <pre>git add backend/app/core/security.py backend/tests/test_security.py frontend/.env.example database/seeds/demo_data.sql git commit -m "Accept ES256 Supabase tokens; document API URL"</pre>
2	Switch on live updates in the database MUST	The app listens for changes to orders, stock, notifications and listings, but Supabase only sends changes for tables that are switched on, and none are. Without this, a supplier confirming an order will not appear on the shop's phone until someone pulls to refresh. Make a new file <code>database/migrations/0019_realtime.sql</code> (never edit old migrations) and run it in the SQL editor: <pre>alter publication supabase_realtime add table orders, stock_items, notifications, supplier_listings;</pre>
3	Empty the Firebase setting MUST	<code>FIREBASE_CREDENTIALS_PATH</code> in <code>backend/.env</code> points to a file that does not exist. When the backend tries to send a phone notification, it will crash. Push notifications are a later phase, so leave the value blank: <pre>FIREBASE_CREDENTIALS_PATH=</pre>
4	Turn off "Confirm email" in Supabase MUST	Authentication → Providers → Email. While it is on, a new user is not signed in until they click an email link, so the profile-setup screen fails. The free email sender also only sends a few emails per hour.
5	Change the database password MUST	It was once written into the app's settings file, so we treat it as exposed. Settings → Database → Reset password. Nothing in our code uses it, so nothing breaks.
6	Start both sides the right way	Backend (in <code>backend/</code>, venv switched on): <pre>.venv\Scripts\activate uvicorn app.main:app --reload --host 0.0.0.0 --port 8000</pre> App (in <code>frontend/</code>): <code>npx expo start --clear</code>. <code>--clear</code> makes Expo read the <code>.env</code> file again.

- 7 **Go / no-go test** **MUST** Log in as `wasantha.kade@inventix.lk` . **Pass** = no "sign-in could not be verified" message, and the Stocks screen shows the **4 demo products** (rice, milk powder, sugar, biscuits). If it fails, stop and fix this before anything else.
- 8 **Tell the team how to connect** We all share one Supabase project. Send the `.env` values and the two demo passwords **privately** (never through GitHub). Decide whose laptop runs the backend. Everyone's `EXPO_PUBLIC_API_URL` must point to that laptop's current IP (`ipconfig`).

The IP address changes. `172.20.10.9` is a phone-hotspot address. If you reconnect, it can change. When every screen suddenly shows "Network request failed", check `ipconfig` and update `EXPO_PUBLIC_API_URL` , then restart Expo with `--clear` .

3. The order of work

We start with the parts that everything else needs (login), then the shop owner's side, then the supplier's side, and finally the full order journey using both accounts.

Phase	Feeds	Why this order
0	Checklist (section 2)	Nothing works properly until these are done.
1 — Shared	Auth, Catalog	Every other feed needs a signed-in user with a role. Adding stock and listings both need catalog search.
2 — Customer	Stocks → Suppliers → Ordering → Delivery → Uploads → Reports	Each one uses the one before it: you need stock to reorder, a supplier to order from, and an order to track.
3 — Supplier	Listings → Orders → Supplier delivery	The supplier's side of the orders the customer sent in Phase 2.
4 — End to end	Full order journey, live updates, notifications	An order passes between both roles many times. It can only be tested once both sides work.
5 — Finish	Docs, pull request, merge	Make <code>main</code> the working version for the whole team.

Suggested timetable

Day	Work
Thu 11 Sept	Phase 0, Phase 1, Stocks
Fri 12 Sept	Suppliers, Ordering, Delivery, Uploads, Reports
Sat 13 Sept	Phase 3 (supplier), Phase 4 (end to end), Phase 5 (merge)
Sun 14 Sept	Demo practice only. No new features.

Why customer first — and one thing to know

The customer side has the most screens, and the demo data already gives it something to show: 4 stock items, 2 supplier listings, one finished order with a rating, and one order in progress ("processing"). So most customer screens can be tested without the supplier.

The exception is the **order journey**. An order moves between the two roles:



Blue = shop's action. Orange = supplier's actions. Green = finished (stock goes up). Red = stopped. Only the shop can set "purchased", because that is when its stock increases.

So in Phase 2 we test what the customer *can* do alone (send, cancel, view). The full journey waits for Phase 4.

4. The routine for every feed

Do the same six steps for each feed. Keep the backend terminal visible: every screen action prints a line there, which is the fastest way to see what happened.

- 1 **Open the screen and watch the backend terminal.** Each request prints a line with the address and a status number, for example `GET /customer/stocks 200`. No line at all means the app did not call the backend (see section 8).
- 2 **Check the status number.** `200` or `201` or `204` = success. `401` = login problem. `403` = wrong account type. `404` = wrong address or item not found. `422` = the app sent data in the wrong shape. `500` = backend crashed (read the red error text in the terminal).
- 3 **Check the numbers against the database.** Open the same table in Supabase (Table Editor) and compare. Example: the Stocks screen must show the same quantities as the `stock_items` table.
- 4 **Try every button on that screen.** After each action, refresh and check the database changed the way you expected.
- 5 **Try one wrong thing on purpose.** Example: order more than the supplier has, or add a product you already track. The app should show a clear message, not crash.
- 6 **Commit.** One commit per feed, so any feed can be undone on its own:

```
git add -A
git commit -m "Integrate stocks feed"
```

Useful tool: the backend's own test page. With the backend running, open `http://localhost:8000/docs` on the laptop. It lists every endpoint and what data it expects. When a screen gets a `422`, compare what the app sent with what this page says the backend wants.

Fix at the root, not on the screen. If a number is wrong, find *where* it is calculated, which is almost always the backend's `service.py` for that feed. Never "correct" a number in the app just to make it look right. The app must only show what the backend sends.

Where things live, per feed

Layer	File
Screen	<code>frontend/app/(customer)/...</code> or <code>frontend/app/(supplier)/...</code>
App's request	<code>frontend/src/api/<feed>.ts</code>
Shape the app expects	<code>frontend/src/types/api.ts</code>
Backend address list	<code>backend/app/feeds/<role>/<feed>/routes.py</code>
Shape the backend sends	<code>backend/app/feeds/<role>/<feed>/schemas.py</code>
The actual logic	<code>backend/app/feeds/<role>/<feed>/service.py</code>

5. Feed by feed

For each feed: the screens, the backend addresses it uses, what to test, and any problem we already know about. All 43 app requests were checked against the backend on 11 September: **every address matches**. The problems below are about the *data* sent, not the addresses.

Phase 1 — Shared

Auth — sign in, register, profile

Screens: login, register, choose-role, profile-setup

Backend: POST /auth/profile , GET /auth/profile (signing in itself goes straight to Supabase)

Test: sign in with both demo accounts; sign out; register a brand-new test account as a shop, fill in profile-setup, and check a new row appears in `profiles` with `role = customer` .

Known problems: **FIX** If an account has no profile, the app silently returns to the login screen with no message. **FIX** After profile-setup, the app does not update who is signed in, so some screens may not know the user until the app restarts.

Catalog — product search

Screens: used inside "Add stock" and "Add listing"

Backend: GET /catalog/search?q=...

Test: search "rice", "milk", "sugar"; results must come from the 36 catalog products.

Phase 2 — Customer (log in as the shop)

Stocks — the home screen

Screens: stocks list with pie chart, stock item detail, add stock

Backend (8): GET /customer/stocks , /summary , /seasonal , /{id} , /{id}/adjustments , POST /customer/stocks , PATCH /{id} , POST /{id}/adjust

Test: 4 demo items appear; the pie chart numbers equal the list sections; open an item and see its history; change the low-stock limit; adjust a quantity with a reason and check a new `stock_adjustments` row; add a new product; try adding the same product twice (must be refused).

Known problems: **MINOR** A low-stock limit of exactly 0 is replaced by a calculated default. **MINOR** The "sample data" banner only exists on this screen. It will not show now, which is correct.

Suppliers — find and compare

Screens: suppliers list and search, supplier profile

Backend (3): `GET /customer/suppliers?q=...`, `/by-product/{id}`, `/id`

Test: Lanka Traders appears; search by name; open the profile and see its listings and past orders.

Known problems: **MINOR** The ranking score may be empty until the daily ranking job has run. This is expected, not a bug.

Ordering — the restock popup

Screens: restock popup on the Stocks screen

Backend (2): `POST /customer/ordering/message`, `POST /customer/ordering/send`

Test: pick low items, choose Lanka Traders, see the message written for you, send by "in app"; a new order appears in Delivery and in the `orders` table; tap Send twice quickly: only **one** order may be created.

Known problems: **MUST FIX FIRST** The backend returns the message as an object (`{ message_body, problems }`), but the app expects plain text. The popup would show nonsense. Fix in `frontend/src/api/ordering.ts`. **FIX** If the delivery date box is typed in and then cleared, the app sends an empty text, which the backend refuses (422). It should send nothing instead. **MINOR** WhatsApp and email are not set up, so those channels still create the order but say the message was not sent.

Delivery — tracking orders (customer)

Screens: delivery list, order detail, history

Backend (5): `GET /customer/delivery`, `/id`, `POST /id/confirm-receipt`, `/id/cancel`, `/id/advance`

Test: the "processing" demo order shows with its progress bar; the "purchased" order is in history with its 5-star rating; cancel the order you just sent (allowed only while "requested").

Later: confirming receipt needs the supplier to mark the order delivered first, so it is tested in Phase 4.

Uploads — POS sales report

Screens: upload, column matching, unmatched products

Backend (5): `POST /customer/uploads`, `/id/mapping`, `GET/POST /id/unmatched`, `POST /id/apply`

Test: upload a sample CSV; columns are guessed; link any unknown product names; apply; stock goes down and `stock_adjustments` rows appear; upload the same file again: must be refused.

Known problems: **MUST FIX FIRST** The app sends only the file's **name**, but the backend needs the file itself (as a "multipart" upload). Both `api/uploads.ts` and `api/client.ts` (which always says "JSON") need changing. **NEEDED** A sample POS report CSV using catalog product names.

Reports

Screens: reports list, report detail

Backend: GET /customer/reports

Test: the 5 report sections appear. They have no figures yet. That is by design, since there is no sales history.

Known problems: **MINOR** The backend also says whether each report is available; the app ignores this.

Phase 3 — Supplier (log in as the supplier)

Listings — what the supplier sells

Screens: listings list, listing detail, add listing

Backend (4): GET /supplier/listings , /{id} , POST /supplier/listings , PATCH /{id}

Test: 2 demo listings appear (rice, milk powder); add a new product; change a price; switch a listing off and on.

Orders — incoming orders

Screens: orders list, order detail

Backend (4): GET /supplier/orders , /{id} , POST /{id}/confirm , /{id}/reject

Test: the order the shop sent in Phase 2 appears as new; confirm it; send another and reject it (a reason of at least 5 characters is required).

Supplier delivery — moving orders forward

Screens: delivery queue

Backend: POST /supplier/delivery/{id}/advance , /{id}/delivered

Test: move a confirmed order through processing → put to delivery → on the way → delivered. Skipping a stage must be refused.

Note: **MINOR** GET /supplier/delivery exists in the backend but the app does not use it (it groups orders itself). Not a problem.

Shared — Notifications

Notifications

Screens: notifications

Backend (3): `GET /notifications`, `POST /{id}/read`, `POST /notifications/device`

Test: after order actions, a notification appears in the list; tapping it marks it read. Phone pop-up (push) notifications are switched off until Firebase is set up. That is expected.

6. Phase 4 — The full order journey

Use two phones (one signed in as the shop, one as the supplier), or one phone and sign out / in. Do the whole journey once from start to end. **This is exactly what we will show on demo day**, so it is also demo practice.

#	Who	Action	Check
1	Shop	Stocks → reorder the low items from Lanka Traders, in app	Order is "requested"
2	Supplier	Orders → the new order has appeared without refreshing	Live updates work
3	Supplier	Confirm	Shop's screen shows "confirmed" by itself
4	Supplier	Delivery → processing → put to delivery → on the way	Shop's progress bar moves each time
5	Supplier	Mark delivered	Shop gets a notification
6	Shop	Delivery → confirm receipt → give a rating	Order is "purchased"
7	Shop	Stocks	Quantities went up by the ordered amounts; the items left "low stock"
8	Shop	Suppliers → Lanka Traders	The new rating is included
9	Shop	Upload the sample POS report	Quantities go down by the sold amounts

If all nine steps pass, integration is finished. Every feed, all three connections, the security checks and live updates have been used for real.

Phase 5 — Finish

1. **Update the docs.** `docs/10-connections.md` ("What is connected today") and `docs/05-changelog.md` still describe the project as not connected. `database/README.md` should say migrations run to 0019.
2. **Push the branch** and open a pull request into `main` :

```
git push -u origin feature/backend-integration
```
3. **One teammate reviews** the pull request and runs the Phase 4 journey on their own phone.
4. **Merge** into `main`. Everyone runs `git pull` on `main`.
5. **After demo day, not before:** delete the fixture (fake) data from `frontend/src/api/*.ts`. Careful: `REPORT_SECTIONS` and `CATALOG` are used by real screens, so remove those last, or keep them.

Git rules for this work

- **One branch** (`feature/backend-integration`), **one commit per feed**. No branch per feed: it adds merging work and gives nothing extra.
- **Never commit** a `.env` file. Both are ignored by git already. Check `git status` before every commit anyway.
- **Never commit real UUIDs** into `demo_data.sql` . The placeholders stay.
- **Never edit an old migration**. A database change is always a new numbered file.

7. All known problems in one list

Found by checking every app request against the backend on 11 September 2026.

Level	Feed	Problem	Where to fix
MUST	Setup	Live updates not switched on for any table	New <code>0019_realtime.sql</code>
MUST	Setup	Firebase path points to a missing file; sending a push would crash	<code>backend/.env</code> (leave blank)
MUST	Ordering	Message comes back as an object, app expects text	<code>frontend/src/api/ordering.ts</code>
MUST	Uploads	App sends the file name, not the file	<code>api/uploads.ts</code> , <code>api/client.ts</code>
FIX	Auth	No profile → silent return to login, no message	<code>stores/authStore.ts</code>
FIX	Auth	After profile-setup the app does not record who is signed in	<code>profile-setup.tsx</code> / <code>authStore.ts</code>
FIX	Ordering	Cleared date box sends empty text → 422	<code>frontend/src/api/ordering.ts</code>
MINOR	Stocks	Low-stock limit of 0 replaced by a default	<code>stocks/service.py</code>
MINOR	Suppliers	Order history in supplier profile has no fixed shape	<code>suppliers/schemas.py</code>
MINOR	Reports	"Available" flag sent but not used	<code>types/api.ts</code>
MINOR	Supplier delivery	One backend endpoint is never called	None needed
MINOR	Docs	Three docs describe the old, unconnected state	<code>docs/</code> , <code>database/README.md</code>

MUST = the feature does not work until fixed. **FIX** = works in the normal case, fails in a real situation. **MINOR** = safe to leave for the demo.

8. When something breaks

The symptom usually tells you where to look before you open any file.

What you see	Most likely cause
"Network request failed" on every screen	Backend not running, started without <code>--host 0.0.0.0</code> , or the laptop's IP changed. Check <code>ipconfig</code> .
Screens show nothing and no line appears in the backend terminal	The app did not pick up <code>.env</code> . Restart with <code>npm run expo start --clear</code> .

"Your sign-in could not be verified"	Token check failed: <code>core/security.py</code> , or an old session. Sign out and in again.
"Your session has expired"	Normal after a long time. Sign in again.
Everything returns 403	Logged in with the wrong account type for that screen.
One button returns 422	The app sent a field with the wrong name or type. Compare <code>api/<feed>.ts</code> with <code>schemas.py</code> , or use <code>/docs</code> .
500, red error text in the terminal	Read the last lines of the error. A "violates ... constraint" message means the database refused the data.
"The database is not connected yet"	<code>SUPABASE_URL</code> or keys are empty in <code>backend/.env</code> .
Data correct, but only after pulling to refresh	Live updates: check Phase 0 step 2 was run.
Login works, back to the login screen straight away	That account has no <code>profiles</code> row.
You can see another shop's data	Stop. A security rule in <code>database/policies/</code> is wrong. This is the only emergency on this list.

9. Working as a team of four

- **One Supabase project for all of us.** The data, the two demo accounts and the catalog are shared. Nobody runs the database SQL again. Running `demo_data.sql` twice would create duplicate orders.
- **Secrets travel privately.** `.env` values and the demo passwords are sent by private message, never through GitHub. The repository is public.
- **One backend at a time is enough.** Everyone's app can use the same laptop's backend, as long as all phones are on the same network and use that laptop's IP.
- **New test users** are made through the app's Register screen, not SQL. The UUIDs in `demo_data.sql` were only needed once, when the database was empty.
- **Who is doing what.** If more than one person works on integration, split by feed, and tell each other before touching shared files (`api/client.ts` , `types/api.ts` , `dependencies.py`).

10. For demo day

- **Everything shown is real.** Every number on screen comes from the database through the backend. The fake fixture data can no longer appear while `EXPO_PUBLIC_API_URL` is set.
- **Practise the Phase 4 journey** on the actual phones and network you will use.
- **Check the IP address** on the morning of the demo. The hotspot may give a new one.
- **If a marker asks about the catalog:** it is one shared product list so shops and suppliers match exactly; in a real product, suppliers would add new brands, shops would request missing items, and barcodes would stop duplicates.
- **If a marker asks about the POS:** the app reads the sales report file any POS can export, so it works with every POS without a special connection.

Glossary

Word	Meaning
Feed	One area of the app with its own screens and backend addresses, for example Stocks or Delivery.
Endpoint	One backend address the app can call, like <code>GET /customer/stocks</code> . We have 43.
Integration	Making the app use the real backend and checking that the results are right.
Fixture / mock data	Fake data stored inside the app so screens could be built before the backend existed. Now switched off.
Token	A signed pass Supabase gives you at login. The app shows it to the backend with every request.
ES256 / HS256	Two ways of signing tokens. Our Supabase project uses ES256; the backend now accepts both.

Role	Whether an account is a customer (shop) or a supplier . Decides which screens and actions are allowed.
Migration	A numbered SQL file that changes the database structure. Run once, in order, never edited afterwards.
Seed	A SQL file that fills an empty database with starting data (catalog, festivals, settings, demo accounts).
RLS (row level security)	Database rules that decide which rows each user may see, so a shop only sees its own stock.
Live updates (realtime)	Supabase tells the app when a table changes, so the screen refreshes by itself.
Idempotency key	A one-time code sent with "Send order". If the same request arrives twice, the backend creates only one order.
Status code	The number the backend answers with: 2xx success, 401 not signed in, 403 not allowed, 404 not found, 422 wrong data, 500 crash.
LAN IP	The laptop's address on the local network (<code>ipconfig</code>). Phones use it to reach the backend.
Commit / branch / pull request	A saved step in git / a separate line of work / a request to add that work into <code>main</code> .